

Unit 6 Data Types and Operators

Marks: 10

Data type: Data type is a term that is used to show the kind of data values or the type of data that is expected to be handled. There are different data types available in Python:

Numeric Type: Integer, Complex Number, Float

Boolean

Sets

None

Sequence Type: String, List, Tuple

Mapping: dictionary

Numeric: This data type stores only numeric value in Python. It is further classified into three different types:

- i. **Integers:** Integers are the whole numbers having different storage range supported by particular machine. Integers have no fractional parts. They can be positive or negative. e.g. 2, 15, -12 etc.
- ii. **Floating Point Numbers:** The float data type in Python designates a floating point number. Float values are specified with a decimal point. The decimal point signals that is a floating point number not an integer. e.g. 23.65, 458.908
- iii. **Complex Numbers:** A complex number is a number expressed in the form $a+ib$, where, a (real part) and b (imaginary part) are real numbers and i is the imaginary part. Complex number is made up of both real numbers and imaginary numbers. e.g. $2 + 3i$, $1+5i$ etc.

Boolean: Boolean data type is a unique data type, consisting of two values- **True** and **False**

Sequences: A Python sequence is an ordered collection of items, where each item is indexed by an integer. three types of sequence data types are available in Python:

- i. **Strings:** Strings are group of characters. These characters may be alphabets, digit or special character including spaces. String values may be delimited using either single quotes or double quotes, such as: "Basit", 'shahid', '787'
A string in Python can contain as many characters as you want. The string limit is depend on your machine's memory resources. A string can also be empty. In Python, the string type is called str.
- ii. **Lists:** List is a sequence of items separated by commas and the items are enclosed in the square brackets []. Lists are mutable that means they can be change after creation. e.g.

```
a=[1, 2, 4]
b=[]
c=["Basit", 56]
```
- iii. **Tuples:** Tuple is an ordered sequence of items same as list. The only difference is that tuples are immutable and lists are mutable. Tuples once created cannot be modified. Tuples are used to write-protect data and are usually faster than list as it cannot change dynamically. Tuples are defined by parentheses().
e.g. Add=("Hergam", "Bongam", "Alyalpora", "Batapora")

Dictionary: Dictionaries in Python are lists of key : values pairs. This is a very powerful data type to hold a lot of related information that can be associated through keys. The main operation of a dictionary is to extract a value based on the key name. Dictionaries can also be used to sort, iterate

and compare data. Dictionaries are created by using braces {} with pairs separated by comma and key values associated with a colon. In dictionaries, the key must be unique. e.g.

```
dict={'Name' : 'Shahid', 'RollNo': 24, 'Class' : '11th'}
```

```
print(dict.keys())
```

```
print(dict.values())
```

```
print(dict['Class'])
```

output:

```
dict_keys(['Name', 'RollNo', 'Class'])
```

```
dict_values(['Shahid', 24, '11th'])
```

11th

Mutable and Immutable Types: Every variable in Python holds an instance of an object. There are two types of objects in Python:

1. **Mutable Types:** The values of mutable types can be changed after it is created. These are list, dictionary and sets. e.g.

```
a=[10, 20,30,40,50]
```

```
print("Before change")
```

```
print(a)
```

```
a[0]=25
```

```
a[3]=33
```

```
print("After change")
```

```
print(a)
```

output:

```
Before change
```

```
[10, 20, 30, 40, 50]
```

```
After change
```

```
[25, 20, 30, 33, 50]
```

2. **Immutable Types:** The values of immutable cannot be changed after it is created. These are integer, float, Boolean, string and tuple. e.g.

```
a=(10, 20,30,40,50)
```

```
a[0]=25
```

```
a[3]=33
```

```
print(a)
```

output: It will give Error Message: TypeError: 'tuple' object does not support item assignment.

Displaying the data type of a Variable:

You can get the data type of a variable with the type() function.

Example

```
x = 5
```

```
y = "Shahid"
```

```
print(type(x))
```

```
print(type(y))
```

output:

```
<class 'int'>
```

<class 'str'>

Type Conversion in Python:

The process of converting a data type into another data type is known as type conversion. Python has two types of type conversion as follows:

1. **Implicit Type Conversion:** In implicit type conversion, Python automatically converts one data type to another data type. This process does not need user involvement. This conversion takes place during compilation or run time. e.g.

```
a=25
b=14.564
sum=a+b
print(sum)
print("The Data Type of sum is:",type(sum))
```

output:

39.564

The Data Type of sum is: <class 'float'>

2. **Explicit Type Conversion:** In explicit type conversion, user converts the data type of an object to required data type. This type conversion is also called typecasting because the user casts the data type of the object. We use the predefined functions like int(), float(), str() etc., to perform explicit type conversion. e.g.

```
A=234
B="123"
print("Data type of A is", type(A))
print("Data type of B is", type(B))
B=int(B)
print("After Conversion Data type of B is", type(B))
```

output:

Data type of A is <class 'int'>

Data type of B is <class 'str'>

After Conversion Data type of B is <class 'int'>

Operators: An operator is a Symbol that is used to perform an operation on values. The values on which the operators work on is called operands. There are two types of operators in Python: **Unary and Binary operators**. Unary operators are those which require only one operand. Such as, unary plus (+), unary minus (-), Bitwise complement (~), logical not (not). Binary operators are those operators that require two operands. Such as, addition (+), Subtraction (-) etc.

There are various types of Binary Operators in Python programming:

1. **Arithmetic Operators:** These operators are used to do arithmetic operations. The arithmetic operators in Python are:

+	Addition	It adds the value of either sided operands.
-	Subtraction	Subtracts the value of right handed operand from left hand operand's value

* Multiplication Multiplies the value of either sided operands.
 / Division Divides left hand operand by right hand operand.
 % Modulus Divides left hand operand by right hand operand and returns remainder.
 // Floor Division Divides left hand operand by right hand operand and ignored the fractional part of result.
 ** Exponentiation (power) Gives raised to power of number.

Example:

```
a=9
b=2
print("Result of a+b=",a+b)
print("Result of a-b=",a-b)
print("Result of a*b=",a*b)
print("Result of a/b=",a/b)
print("Result of a%b=",a%b)
print("Result of a//b=",a//b)
print("Result of a**b=",a**b)
```

output:

```
Result of a+b= 11
Result of a-b= 7
Result of a*b= 18
Result of a/b= 4.5
Result of a%b= 1
Result of a//b= 4
Result of a**b= 81
```

Logical Operators: These operators are used to make a decision on two conditions. Logical operators are typically used with Boolean values. Logical operators also return Boolean value. They are mainly used to control program flow. The logical operators are:

1. Logical AND: The logical AND operator (and) returns the value True, if both expressions evaluate to True otherwise returns False. For example:

((2<4)and(5<8)) gives result True because both expressions are True.

((2>4)and(5<8)) gives result False because first expressions is False.

The truth table for Logical AND operator is:

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

2. Logical OR: The logical OR operator (or) returns the value True, if either or both expressions are True otherwise returns False. For example:

((2>4)or(5>8)) gives result False because both expressions are False.

((2>4)or(5<8)) gives result True because second expressions is True.

The truth table for Logical OR operator is:

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

- 3. Logical NOT:** This operator is unary operator and works with single operand or single expression. The logical NOT operator reverses the value of its operand or expression. If the expression is True it returns False and if the expression is False it returns True. For example:

not(4==4) gives result False, because expression is True.

not(4==5) gives result True, because expression is False.

The truth table for Logical NOT operator is:

A	not A
True	False
False	True

Relational Operators: Relational operators compare two operands to one another. The relational operators are ==, !=, <, >, <=, >=. Relational operators return Boolean value when used in an expression. Relational operators are also called Comparison operators. The expressions which use relational operators are called Relational expression. The Relational operators in python programming are:

== Equal to Checks if the values of two operands are equal or not. If yes then condition becomes True otherwise False.

!= Not equal to checks if the value of two operands are equal or not, if values are not equal then condition becomes True otherwise False.

> Greater than checks if the value of left operand is greater than the value of right operand, if yes then condition becomes True otherwise False.

< Less than checks if the value of left operand is less than the value of right operand, if yes then condition becomes True otherwise False.

>= Greater than or equal to checks if the value of left operand is Greater than or equal to the value of right operand, if yes then condition becomes True otherwise False.

<= Less than or equal to checks if the value of left operand is Less than or equal to the value of right operand, if yes then condition becomes True otherwise False.

Example:

a=5

b=10

```
print("The Result of a>b is", a>b)
```

```
print("The Result of a>=b is", a>=b)
```

```
print("The Result of a<b is", a<b)
```

```
print("The Result of a<=b is", a<=b)
```

```
print("The Result of a==b is", a==b)
```

```
print("The Result of a!=b is", a!=b)
```

output:

The Result of a>b is False

The Result of a>=b is False

The Result of a<b is True

The Result of a<=b is True

The Result of a==b is False

The Result of a!=b is True

Assignment Operators: These Operators are also called Shorthand operators or Augmented assignment Operators. These operators are used to assign the value of an expression to a variable.

+= Add AND assignment operator, it adds right operand to the left operand and assigns the result to left operand.

-= Subtract AND assignment operator, it subtracts right operand from left operand and assigns the result to left operand.

***=** Multiply AND assignment operator, it multiplies right operand with left operand and assigns the result to left operand.

/= Divide AND assignment operator, it divides left operand with the right operand and assigns the result to left operand.

`%=` Modulus AND assignment operator, it divides left operand with right operand and assigns the remainder to the left operand.

`**=` Exponentiation AND assignment operator, it gives left operand raised to the power of right operand and assigns the value to left operand.

`//=` Floor Division AND assignment operator, it floor divides left operand with the right operand and assigns the result to the left operand.

Example:

```
a=5
```

```
b=2
```

```
print(" a=",a, " b=",b)
```

```
a+=b
```

```
print("The Result of a+=b is", a)
```

```
a=5
```

```
b=2
```

```
a-=b
```

```
print("The Result of a-=b is", a)
```

```
a=5
```

```
b=2
```

```
a*=b
```

```
print("The Result of a*=b is", a)
```

```
a=5
```

```
b=2
```

```
a/=b
```

```
print("The Result of a/=b is", a)
```

```
a=5
```

```
b=2
```

```
a%=b
```

```
print("The Result of a%=b is", a)
```

```
a=5
```

```
b=2
```

```
a//=b
```

```
print("The Result of a//=b is", a)
```

```
a=5
```

```
b=2
```

```
a**=b
```

```
print("The Result of a**=b is", a)
```

output:

```
a= 5  b= 2
```

The Result of a+=b is 7

The Result of a-=b is 3

The Result of a*=b is 10

The Result of a/=b is 2.5

The Result of a%=b is 1

The Result of a//=b is 2

The Result of `a**=b` is 25

Python Identity Operators:

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location. There are two Identity Operators in Python:

`Is` Returns True if both variables are the same object

`is not` Returns True if both variables are not the same object

Example:

```
a = ["pear", "apple"]
```

```
b = ["pear", "apple"]
```

```
c = a
```

```
print(a is c)
```

```
# returns True because c is the same object as a
```

```
print(a is b)
```

```
# returns False because a is not the same object as b, even if they have the same content
```

```
print(a == b)
```

```
# to demonstrate the difference between "is" and "==": this comparison returns True because  
a is equal to b.
```

```
x = ["apple", "banana"]
```

```
y = ["apple", "banana"]
```

```
z = x
```

```
print(x is not z)
```

```
# returns False because z is the same object as x
```

```
print(x is not y)
```

returns True because x is not the same object as y, even if they have the same content

```
print(x != y)
```

to demonstrate the difference between "is not" and "!=": this comparison returns False because x is equal to y

Membership Operators: Those operators are used to validate the membership of a value in a string. These operators are used to find out whether a value is a member of a string or not. There are two membership operators for strings in Python as follows:

1. **in operator:** This operator is used to check if a value exists in a sequence or not. It evaluates True if it finds a variable in the specified sequence and False otherwise.

```
>>>a=["Basit", "Good", "Morning"]  
>>>"Good" in a  
>>>True  
>>>"Bad" in a  
>>>False
```
2. **not in operator:** This operator is the opposite of 'in' operator. So, if a value does not exist in the sequence, then it will return a True else it will return a False.
Example:

```
>>>a=["Basit", "Good", "Morning"]  
>>>"Good" not in a  
>>>False  
>>>"Bad" not in a  
>>>True
```

Tariq Sir